

Nectar Data Engineer Challenge — Submission Report

Candidate: Venkatesh · **Role:** Data Engineer · **Repository:** nectar-iot-data-platform

Page 1 — Summary and approach

What was built

A working IoT lakehouse: Kafka ingress, a Delta Lake medallion (bronze → silver → gold), a declarative data quality framework, a dimensional model with SCD2 and an asset-topology closure table, Databricks Workflows orchestration, and a Spark Structured Streaming path. Every task in the brief has running code behind it, not a description.

Verified run (2 cores, laptop-scale): 608,261 telemetry rows and 3,624 events through bronze → silver → gold → hierarchy → quality report in **103 seconds**; 35 quality rules evaluated; 11,946 rows quarantined (1.95%); 0 blocking breaches → verdict **PASS; 35/35 unit tests pass**. The outputs are committed under docs/results/.

The approach

The challenge ships no data, so the first thing built was a **generator that injects defects at known rates** — duplicates, nulls, physically impossible values, unparseable and implausible timestamps, unknown asset ids, late arrivals, unparseable payloads — plus known-position anomalies: 6 degraded assets, 4 devices that go silent, 2 site-wide consumption excursions.

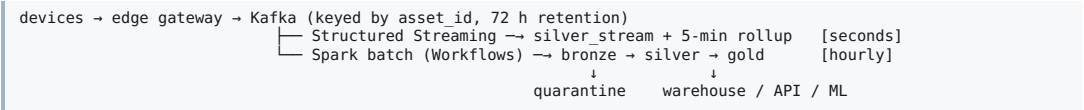
That inverts how the rest of the submission can be read. The quality framework does not merely *claim* to catch bad data; it independently rediscovers the exact counts the generator recorded. The SQL queries do not merely *run*; Q3 returns the six degraded assets, Q4 returns the four dark devices, and Q6 flags exactly the two injected site excursions on the day they were injected. Every number in this report is checkable by re-running `make all`.

Guiding principles

Principle	Consequence in the code
Never lose evidence	Quarantine with rule ids and batch id; bronze keeps malformed rows verbatim
Retries must be safe	Every write is idempotent — MERGE on the business key, dynamic partition overwrite
Stale beats wrong	A blocking quality breach fails the gate task; downstream never runs
One definition of valid	The same <code>Rule</code> objects run in batch and in streaming
Absence is a defect	Freshness and completeness are computed separately — no row-level rule can validate a record that never arrived
Config, not constants	Every threshold, path, partition column and window lives in <code>config/pipeline.yaml</code>

Page 2 — Architecture and data model

Architecture (Task 1)



Two paths, one source of record. The stream answers “what is happening right now”; the batch owns “what actually happened” and restates whatever the stream approximated, including days that received late records. Reconciliation is the design, not a nightly surprise.

Component choices, briefly. Kafka because devices are unreliable and retention makes replay the default recovery mechanism. Delta because two requirements — correcting late records in place, and a streaming writer plus a batch reader on the same table — are impossible on plain Parquet. Spark because one engine in two modes lets the validation rules exist once; Flink would be lower latency but would mean maintaining every business rule twice, and the requirement is seconds, not milliseconds. **Databricks Workflows** for scheduling, because the tables, the transformation graph and the ingestion bookmarks already live in Databricks and a second scheduler would duplicate all of it; every task is a re-runnable notebook, so an external orchestrator can drive the same units if the estate later grows a non-Databricks system.

Scale. Today ~86 M readings/day; the design target is 100× (~8.6 B/day, ~2.5 TB raw). Kafka scales on partitions (6 → ~120), streaming state is bounded by the watermark (~4.5 M keys at 100×, RocksDB-backed), and the batch job is partition-scoped so runtime tracks batch size, not history. At 200 GB/day,

sub-partitioning by `site_id` becomes worthwhile — the partition columns come from config specifically so that is a config edit.

Data model (Task 3)

Star schema: `dim_date`, `dim_time`, `dim_site`, `dim_building`, `dim_asset` (**SCD Type 2**), `dim_asset_hierarchy` + `asset_closure`; facts `fact_telemetry` (one row per reading), `fact_energy_hourly` (asset × hour), `fact_event` (one row per event).

SCD2 is not decoration. Assets are relocated between buildings and re-rated after retrofits. Type 1 would silently restate last quarter's per-building energy the moment a chiller moved. A SHA-256 hash over the tracked attributes drives change detection; `valid_from` / `valid_to` / `is_current` keep history joinable.

Partitioning: `event_date` on facts (every predicate is an event-time range, and late-arrival restatement becomes a partition-scoped overwrite); `ingest_date` on bronze (replay and retention are about when data landed); dimensions unpartitioned and broadcast. Hourly partitioning was rejected — 24× the partitions for the same bytes means small files and slow metadata scans.

Indexing: Delta has no secondary indexes, so — partition pruning, then `ZORDER BY (asset_id, timestamp)` to co-locate one asset's rows, then column ordering (Delta keeps stats on the first 32 columns), then bloom filters on `sensor_id/event_id`. The PostgreSQL mirror uses BRIN on the timestamp, B-tree on (`asset_id`, `ts DESC`), and partial indexes on `is_fault` — plus a unique partial index enforcing exactly one live SCD2 row per natural key.

Page 3 — Pipeline, quality and hierarchy

Pipeline (Task 2)

Bronze lands data verbatim — every column a string — plus lineage (`_ingest_id`, `_source_file`, `_payload_hash`). If a gateway sends `"temperature": "not-a-number"`, that is what bronze holds, so the failure is reproducible and the row is replayable after the fix.

Silver casts (with `try_cast`, so a bad value becomes NULL rather than killing the batch), enriches from the asset register, runs the rule set, splits clean from quarantine, and MERGES on (`asset_id`, `sensor_id`, `timestamp`).

Gold builds dimensions, facts, four curated marts (hourly energy, daily utilisation, daily environment, fault statistics) and three roll-ups (asset, building, site).

Two calculations worth defending.

Energy. Devices report kW, not kWh. The naive `avg(power) × hours` under-counts when readings are lost and double-counts when an asset has two sensors. Instead: collapse to asset grain with `avg` (power is asset-level but arrives per sensor), weight each reading by the gap to the next one capped at 2× the sampling interval (so a 6-hour outage is billed as 10 minutes at the last known load, not 6 hours), then sum `power × duration`. Three unit tests pin this with hand-computable numbers.

Utilisation. Productive hours over **observed** hours, not over 24. Dividing by the calendar day conflates “the asset was idle” with “the asset stopped reporting”. `data_coverage_pct` reports the difference — and Q6 uses it to refuse to compare a partial day with a complete one, which removes an entire class of false-positive anomalies.

Data quality (Task 5)

Rules are **data**: a named row-level predicate plus dimension, severity, threshold and description. That buys three things — the same objects run in batch and streaming, every evaluation writes an auditable row to `dq_results`, and adding a rule means appending to a list.

35 rules across the six DAMA dimensions. **BLOCKING** quarantines the row and fails the batch above threshold; **WARN** keeps and flags it. The split is about *usability*: a null `asset_id` makes a reading unattributable (**BLOCKING**), a null `temperature` is a gap in one series while the power reading remains perfectly usable (**WARN**).

Outliers get two layers, because a fixed range cannot tell that a chiller at 30 °C is broken while a boiler at 30 °C is merely cold: hard physical ranges from config, plus a **modified z-score** ($0.6745 \times (x - \text{median}) / \text{MAD}$) against each asset's own baseline. MAD rather than standard deviation, because a handful of stuck-sensor 9999s would inflate σ enough to mask themselves.

Reporting is automated — JSON for CI and alerting, self-contained HTML (no CDN) for humans, with `_latest` filenames so a dashboard can bookmark one URL.

Reconciliation with the generator:

Injected	Generator	Detected
Unknown asset ids	393	393
Broken timestamps	589	330 unparseable + 259 implausible
Out-of-range values	786	786
Silent devices	4	4 stale
Unparseable payloads	481	481 (DLQ)

Asset hierarchy (Task 4)

`parent_asset_id` is an adjacency list; every interesting question about it is a recursive traversal, and Spark SQL 3.5 has no `WITH RECURSIVE`. So the **transitive closure** is materialised once per batch — `asset_closure(ancestor_id, descendant_id, depth, path)` — turning every hierarchy query into one indexed join and every subtree roll-up into one group-by. 85 assets → 136 closure rows, max depth 2, sub-second.

A **property graph** (NetworkX in-process, plus a complete Neo4j schema/loader/query pack) ships alongside, for when relationships gain types beyond containment — *feeds, is monitored by, shares a circuit with* — and the structure becomes a mesh. The tests assert both models return the same answers; a divergence would give an operator two different blast radii.

Two subtleties the implementation handles: **orphans must be detected before the data is cleaned** (silver nulls the dangling pointer, so the verdict is computed at conform time and preserved), and **“disconnected” is not automatically a defect** — a standalone boiler legitimately has no asset parent, so `connectivity_status` separates `STANDALONE` from `ORPHANED` and `UNASSIGNED`.

Page 4 — SQL, orchestration and streaming

SQL challenge (Task 6)

All six queries are in `sql/analytics/`, executed against the gold layer, with results in `data/query_results/`. Beyond returning rows, each takes a position on a question the brief leaves open:

Q1 — top 10 by energy. Reads the hourly energy fact, so the duration-weighted integration already happened. Adds `load_factor_pct` (average power ÷ nameplate), because a high-kWh asset with a low load factor is *oversized*, not *overworked* — a different remediation.

Q2 — average daily energy per site. Reports two denominators, because they disagree and answer different questions: mean over *active* days (what a working day costs) and total ÷ *calendar* days (what to bill). Quoting one without the other is how energy reports go wrong after an outage. Weekday/weekend split included, since HVAC load is occupancy-driven.

Q3 — >10 faults in 30 days. Anchored to the latest event in the data, not `CURRENT_DATE`: anchoring a backfill to wall-clock time silently returns nothing. Filters on the partition column so the engine prunes rather than scans.

Q4 — silent for 24 h. The trap: this is about records that *do not exist*, so it cannot be a filter over telemetry. It starts from the asset register and LEFT JOINS the last-seen watermark, and distinguishes `SILENT` (reported before, stopped) from `NEVER_REPORTED` (a commissioning gap) — different teams fix those.

Q5 — hourly utilisation per building. Computed from the atomic fact with a `LEAD` window, because the correct calculation is time-weighted; a naive `COUNT(RUNNING)/COUNT(*)` is wrong whenever sampling is irregular.

Q6 — abnormal power increases. Two detectors, either can fire: a robust z-score against the site’s own trailing 7-day baseline (excluding today, so the anomaly cannot raise its own threshold), and a same-weekday week-over-week comparison (HVAC load is strongly weekly — comparing Saturday to Friday manufactures anomalies). The z-score detector is paired with a minimum effect size, because a site so stable that a 0.5% move is statistically extreme still does not need an engineer. Partial days are excluded via `data_coverage_pct`.

On the shipped dataset Q6 returns exactly 4 rows: `SITE-CBE` and `SITE-SIN` on **2026-08-14** — the two injected excursions on their injection date — and the same two sites on the 15th as the excursion tails off. Both are caught by the week-over-week arm, precisely because the 15th is a low-load Saturday whose drop against the mixed weekday baseline would have fooled the z-score arm.

Orchestration (Task 7)

Three Databricks Workflows jobs, defined as JSON in version control. `nectar-iot-batch` runs hourly: pipeline → quality gate → hierarchy → serving, with `max_concurrent_runs: 1` because the layers MERGE into shared tables, and `queue.enabled` so an overrunning hour is backfilled rather than dropped. `nectar-iot-maintenance` runs nightly and separately, so a compaction failure can never fail the pipeline that produces the data: restate late-affected partitions **first**, then OPTIMIZE, then VACUUM, then prune quarantine — order matters so compaction runs once over final files. `nectar-iot-streaming` is a continuous job, because a stream never completes and therefore can never be a predecessor in a dependency graph.

Retries are 1–3 with backoff — safe only because every task is idempotent; without that, automatic retries

are a data-duplication mechanism. The quality gate is set to `max_retries: 0` deliberately, to *skip* retries: a data quality breach is not transient. An overrunning hour queues rather than being dropped. SLA miss warns via a `RUN_DURATION_SECONDS` health rule (dashboards stale); failure pages (dashboards wrong), with the `run_id` and a ready-to-run quarantine query in the alert.

Real-time pipeline (Bonus Option A)

Kafka → parse → DLQ for unparseable payloads → stream-static join to the asset register → **the same rule objects the batch uses** → watermarked dedupe → `silver.telemetry_stream` + a 5-minute rollup.

Event time throughout, not processing time. The 15-minute watermark is the key knob: too short drops late readings, too long grows state without bound. Records later than that are not lost — they fall into the batch pipeline's late-arrival path, which is why both exist. Exactly-once comes from the checkpoint plus Delta's transactional commit (offsets and data commit together); `dropDuplicates` inside the watermark additionally absorbs at-least-once gateway delivery.

Verified run over 608k records: 596,400 clean rows, 5,460 quarantined, **484 DLQ** (exactly the injected unparseable payloads), 481,654 rollup windows. It runs against a file source with no broker, which is how the tests and the demo exercise it.

Page 5 — Results, trade-offs and next steps

Results

Metric	Value
Telemetry rows processed	608,261
Events processed	3,624
End-to-end batch runtime (2 cores)	103 s
Gold tables produced	18
Quality rules evaluated	35
Rows quarantined	11,946 (1.95%)
Blocking breaches	0 → verdict PASS
Closure table rows	136 (85 assets, max depth 2, 6 orphans)
Streaming run	596,400 clean · 5,460 quarantined · 484 DLQ · 481,654 windows
Unit tests	35 passing
Ground truth recovered	Q3 = the 6 degraded assets; Q4 = the 4 silent devices; Q6 = the 2 injected site excursions

Honest trade-offs

Delta vs a Parquet fallback. The submission targets Delta and defaults to it. A thin storage layer (`io_layer.py`) also supports plain Parquet, because Delta's jars are fetched from Maven Central at session start and some environments cannot reach it. The fallback is explicitly dev-only — no atomic commits, no MERGE, no time travel, no safe concurrent writers — documented as a capability table, and `resolve_format()` warns loudly if it has to downgrade. It exists so the project is runnable everywhere, not because Parquet is adequate.

Spark Streaming vs Flink. Flink is genuinely better for low-millisecond, per-event state. The requirement is seconds, and a second engine means every business rule implemented twice. One codebase for both paths was worth more than the latency difference.

Closure table vs graph database. The closure is right for a read-heavy analytics platform with a tree-shaped, slowly-changing topology. It stops being right once relationships are typed and the structure becomes a mesh. Both are implemented; the tests keep them honest.

Workflows vs an external orchestrator. Workflows wins here because the pipeline is already inside Databricks: serverless compute, lineage that joins job runs to table history to quality metrics, continuous mode for the streams, and SLA as a declarative health rule rather than a callback to maintain. An external orchestrator earns its place only when a dependency crosses out of the platform — an on-prem extract, a vendor SFTP drop — and it would then *trigger* these jobs rather than replace them. Of those, Dagster fits a medallion lakehouse better than Airflow: assets rather than tasks, quality checks first-class. Airflow DAGs for the same graph are kept in `alternatives/airflow/` so the choice is reversible.

Batch interval. Hourly, with the stream covering sub-hour latency. Going sub-hourly on the batch would multiply small files and scheduling overhead for latency the stream already provides.

What I would do next

1. **dbt for the gold layer.** The curated marts and roll-ups are SQL-shaped; dbt gives lineage, docs and tests for free, with Spark keeping bronze/silver.
2. **Great Expectations / Soda.** The rule engine here is deliberately small and dependency-free; a shared

expectation catalogue is worth adopting once more than one team writes rules.

3. **Predictive maintenance.** The scaffolding is in place — atomic grain, SCD2 for point-in-time correctness, fault labels, `health_score`, and `is_late_arrival` as a leakage guard.
4. **REST API and Streamlit dashboard** (Bonus B and C) on the same gold layer.
5. **Iceberg evaluation**, if Nectar's customers stop being Spark-centric.

Closing

The parts of this submission I would defend hardest are not the component choices — those are conventional and would be conventional at most companies. They are the decisions where the obvious implementation is quietly wrong: integrating power into energy without double-counting multi-sensor assets or billing outages; dividing utilisation by observed rather than calendar time; detecting silent devices from the register rather than from the telemetry; catching orphans before the cleaning step erases them; and refusing to compare a partial day with a complete one. Each of those produces a number that looks perfectly reasonable and is wrong, which is the failure mode that matters most in a platform whose output people act on.

Full detail: `docs/01_architecture.md` · `02_data_model.md` · `03_asset_hierarchy.md` · `04_data_quality.md` · `05_orchestration.md`. Reproduce everything with `make all`.